

Single-Source Shortest Paths

Dr. Bugra Caskurlu¹

¹ Artificial Intelligence Engineering Department
TOBB ETU University

January 30, 2024

Introduction

Motivation and Definitions

- Suppose you are given a road map of Turkey on which distances between pairs of adjacent intersections are marked, and asked to find the shortest route from Edirne to Antalya.

Introduction

Motivation and Definitions

- Suppose you are given a road map of Turkey on which distances between pairs of adjacent intersections are marked, and asked to find the shortest route from Edirne to Antalya.
- One possible way would be to enumerate all the routes, add up the distances on each route, and select the shortest. But even if you disallow routes that contain cycles, there are an enormous number of worthless possibilities, such as a route that passes through Kars.

Introduction

Motivation and Definitions

- Suppose you are given a road map of Turkey on which distances between pairs of adjacent intersections are marked, and asked to find the shortest route from Edirne to Antalya.
- One possible way would be to enumerate all the routes, add up the distances on each route, and select the shortest. But even if you disallow routes that contain cycles, there are an enormous number of worthless possibilities, such as a route that passes through Kars.
- In a **shortest-paths problem**, we are given a weighted, directed graph $G = (V, E)$, with weight function $w : E \rightarrow \mathbb{R}$ mapping edges to real-valued weights.

Introduction

Motivation and Definitions

- Suppose you are given a road map of Turkey on which distances between pairs of adjacent intersections are marked, and asked to find the shortest route from Edirne to Antalya.
- One possible way would be to enumerate all the routes, add up the distances on each route, and select the shortest. But even if you disallow routes that contain cycles, there are an enormous number of worthless possibilities, such as a route that passes through Kars.
- In a **shortest-paths problem**, we are given a weighted, directed graph $G = (V, E)$, with weight function $w : E \rightarrow \mathbb{R}$ mapping edges to real-valued weights.
- The **weight** $w(p)$ of path $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its edges:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

Introduction

Motivation and Definitions

- Suppose you are given a road map of Turkey on which distances between pairs of adjacent intersections are marked, and asked to find the shortest route from Edirne to Antalya.
- One possible way would be to enumerate all the routes, add up the distances on each route, and select the shortest. But even if you disallow routes that contain cycles, there are an enormous number of worthless possibilities, such as a route that passes through Kars.
- In a **shortest-paths problem**, we are given a weighted, directed graph $G = (V, E)$, with weight function $w : E \rightarrow \mathbb{R}$ mapping edges to real-valued weights.
- The **weight** $w(p)$ of path $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its edges:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

- The **shortest path weight** $\delta(u, v)$ is defined as:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \overset{p}{\rightsquigarrow} v\} & \text{if there is a path from } u \text{ to } v, \\ \infty & \text{otherwise.} \end{cases}$$

Introduction

Motivation and Definitions

- Suppose you are given a road map of Turkey on which distances between pairs of adjacent intersections are marked, and asked to find the shortest route from Edirne to Antalya.
- One possible way would be to enumerate all the routes, add up the distances on each route, and select the shortest. But even if you disallow routes that contain cycles, there are an enormous number of worthless possibilities, such as a route that passes through Kars.
- In a **shortest-paths problem**, we are given a weighted, directed graph $G = (V, E)$, with weight function $w : E \rightarrow \mathbb{R}$ mapping edges to real-valued weights.
- The **weight** $w(p)$ of path $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its edges:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

- The **shortest path weight** $\delta(u, v)$ is defined as:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \overset{p}{\rightsquigarrow} v\} & \text{if there is a path from } u \text{ to } v, \\ \infty & \text{otherwise.} \end{cases}$$

- A **shortest path** from u to v is defined as any path p from u to v with weight $\delta(u, v)$.

Problem Definition and Its Variants

Single-Source Shortest Paths Problem and Related Problems

- We will focus on the **single-source shortest-paths problem**: given a graph $G = (V, E)$, we want to find a shortest path from a given **source** vertex $s \in V$ to each vertex $v \in V$.

Problem Definition and Its Variants

Single-Source Shortest Paths Problem and Related Problems

- We will focus on the **single-source shortest-paths problem**: given a graph $G = (V, E)$, we want to find a shortest path from a given **source** vertex $s \in V$ to each vertex $v \in V$.
- The algorithm for the single-source problem can solve many other problems, including the following variants.

Problem Definition and Its Variants

Single-Source Shortest Paths Problem and Related Problems

- We will focus on the **single-source shortest-paths problem**: given a graph $G = (V, E)$, we want to find a shortest path from a given **source** vertex $s \in V$ to each vertex $v \in V$.
- The algorithm for the single-source problem can solve many other problems, including the following variants.
 - 1 **Single-destination shortest-paths problem**: Find a shortest path to a given **destination** vertex t from each vertex v . By reversing the direction of each edge in the graph, we can reduce this problem to a single-source problem.

Problem Definition and Its Variants

Single-Source Shortest Paths Problem and Related Problems

- We will focus on the **single-source shortest-paths problem**: given a graph $G = (V, E)$, we want to find a shortest path from a given **source** vertex $s \in V$ to each vertex $v \in V$.
- The algorithm for the single-source problem can solve many other problems, including the following variants.
 - 1 **Single-destination shortest-paths problem**: Find a shortest path to a given **destination** vertex t from each vertex v . By reversing the direction of each edge in the graph, we can reduce this problem to a single-source problem.
 - 2 **Single-pair shortest-path problem**: Find a shortest path from u to v for given vertices u and v . If we solve the single-source problem with source vertex u , we solve this problem also. Moreover, all known algorithms for this problem have the same worst-case asymptotic running time as the best single-source algorithms.

Problem Definition and Its Variants

Single-Source Shortest Paths Problem and Related Problems

- We will focus on the **single-source shortest-paths problem**: given a graph $G = (V, E)$, we want to find a shortest path from a given **source** vertex $s \in V$ to each vertex $v \in V$.
- The algorithm for the single-source problem can solve many other problems, including the following variants.
 - ① **Single-destination shortest-paths problem**: Find a shortest path to a given **destination** vertex t from each vertex v . By reversing the direction of each edge in the graph, we can reduce this problem to a single-source problem.
 - ② **Single-pair shortest-path problem**: Find a shortest path from u to v for given vertices u and v . If we solve the single-source problem with source vertex u , we solve this problem also. Moreover, all known algorithms for this problem have the same worst-case asymptotic running time as the best single-source algorithms.
 - ③ **All-pairs shortest-paths problem**: Find a shortest path from u to v for every pair of vertices u and v . Although we can solve this problem by running a single-source algorithm once from each vertex, we usually can solve it faster. Additionally, its structure is interesting in its own right.

Problem Definition and Its Variants

Single-Source Shortest Paths Problem and Related Problems

- We will focus on the **single-source shortest-paths problem**: given a graph $G = (V, E)$, we want to find a shortest path from a given **source** vertex $s \in V$ to each vertex $v \in V$.
- The algorithm for the single-source problem can solve many other problems, including the following variants.
 - ① **Single-destination shortest-paths problem**: Find a shortest path to a given **destination** vertex t from each vertex v . By reversing the direction of each edge in the graph, we can reduce this problem to a single-source problem.
 - ② **Single-pair shortest-path problem**: Find a shortest path from u to v for given vertices u and v . If we solve the single-source problem with source vertex u , we solve this problem also. Moreover, all known algorithms for this problem have the same worst-case asymptotic running time as the best single-source algorithms.
 - ③ **All-pairs shortest-paths problem**: Find a shortest path from u to v for every pair of vertices u and v . Although we can solve this problem by running a single-source algorithm once from each vertex, we usually can solve it faster. Additionally, its structure is interesting in its own right.
- In all these problems, edge weights can represent metrics other than distances, such as time, cost, penalties, loss, or any other quantity that accumulates linearly along a path.

Optimal Substructure Property

Importance of the Optimal Substructure Property For a Problem

- Optimal substructure is one of the key indicators that algorithm design techniques such as dynamic programming and the greedy method might be used in solving a problem.

Optimal Substructure Property

Importance of the Optimal Substructure Property For a Problem

- Optimal substructure is one of the key indicators that algorithm design techniques such as dynamic programming and the greedy method might be used in solving a problem.
- Shortest-paths algorithms (Edmonds-Karp maximum-flow algorithm) typically rely on the property that a shortest path between two vertices contains other shortest paths within it.

Optimal Substructure Property

Importance of the Optimal Substructure Property For a Problem

- Optimal substructure is one of the key indicators that algorithm design techniques such as dynamic programming and the greedy method might be used in solving a problem.
- Shortest-paths algorithms (Edmonds-Karp maximum-flow algorithm) typically rely on the property that a shortest path between two vertices contains other shortest paths within it.

Lemma (Subpaths of Shortest Paths are Shortest Paths)

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from vertex v_0 to v_k and, for any i and j such that $0 \leq i \leq j \leq k$, let $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ be the subpath of p from vertex v_i to v_j . Then, p_{ij} is a shortest path from v_i to v_j .

Optimal Substructure Property

Importance of the Optimal Substructure Property For a Problem

- Optimal substructure is one of the key indicators that algorithm design techniques such as dynamic programming and the greedy method might be used in solving a problem.
- Shortest-paths algorithms (Edmonds-Karp maximum-flow algorithm) typically rely on the property that a shortest path between two vertices contains other shortest paths within it.

Lemma (Subpaths of Shortest Paths are Shortest Paths)

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from vertex v_0 to v_k and, for any i and j such that $0 \leq i \leq j \leq k$, let $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ be the subpath of p from vertex v_i to v_j . Then, p_{ij} is a shortest path from v_i to v_j .

Proof

- Let us decompose path p into $v_0 \rightsquigarrow^{p_{0i}} v_i \rightsquigarrow^{p_{ij}} v_j \rightsquigarrow^{p_{jk}} v_k$.

Optimal Substructure Property

Importance of the Optimal Substructure Property For a Problem

- Optimal substructure is one of the key indicators that algorithm design techniques such as dynamic programming and the greedy method might be used in solving a problem.
- Shortest-paths algorithms (Edmonds-Karp maximum-flow algorithm) typically rely on the property that a shortest path between two vertices contains other shortest paths within it.

Lemma (Subpaths of Shortest Paths are Shortest Paths)

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from vertex v_0 to v_k and, for any i and j such that $0 \leq i \leq j \leq k$, let $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ be the subpath of p from vertex v_i to v_j . Then, p_{ij} is a shortest path from v_i to v_j .

Proof

- Let us decompose path p into $v_0 \rightsquigarrow^{p_{0i}} v_i \rightsquigarrow^{p_{ij}} v_j \rightsquigarrow^{p_{jk}} v_k$.
- Assume that there is a path p'_{ij} from v_i to v_j with weight $w(p'_{ij}) < w(p_{ij})$.

Optimal Substructure Property

Importance of the Optimal Substructure Property For a Problem

- Optimal substructure is one of the key indicators that algorithm design techniques such as dynamic programming and the greedy method might be used in solving a problem.
- Shortest-paths algorithms (Edmonds-Karp maximum-flow algorithm) typically rely on the property that a shortest path between two vertices contains other shortest paths within it.

Lemma (Subpaths of Shortest Paths are Shortest Paths)

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from vertex v_0 to v_k and, for any i and j such that $0 \leq i \leq j \leq k$, let $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ be the subpath of p from vertex v_i to v_j . Then, p_{ij} is a shortest path from v_i to v_j .

Proof

- Let us decompose path p into $v_0 \rightsquigarrow^{p_{0i}} v_i \rightsquigarrow^{p_{ij}} v_j \rightsquigarrow^{p_{jk}} v_k$.
- Assume that there is a path p'_{ij} from v_i to v_j with weight $w(p'_{ij}) < w(p_{ij})$.
- Then, $v_0 \rightsquigarrow^{p_{0i}} v_i \rightsquigarrow^{p'_{ij}} v_j \rightsquigarrow^{p_{jk}} v_k$ is a path from v_0 to v_k whose weight is less than $w(p)$, which contradicts that p is a shortest path from v_0 to v_k .

Negative-Weight Edges

Defining Shortest Paths When There are Negative-Weight Edges

- Some instances of the single-source shortest-paths problem may include edges whose weights are negative.

Negative-Weight Edges

Defining Shortest Paths When There are Negative-Weight Edges

- Some instances of the single-source shortest-paths problem may include edges whose weights are negative.
- If the graph $G = (V, E)$ contains no negative-weight cycles reachable from the source s , then for all $v \in V$, the shortest-path weight $\delta(s, v)$ remains well defined, even if it has a negative value.

Negative-Weight Edges

Defining Shortest Paths When There are Negative-Weight Edges

- Some instances of the single-source shortest-paths problem may include edges whose weights are negative.
- If the graph $G = (V, E)$ contains no negative-weight cycles reachable from the source s , then for all $v \in V$, the shortest-path weight $\delta(s, v)$ remains well defined, even if it has a negative value.
- If the graph contains a negative-weight cycle reachable from s , however, shortest-path weights are not well defined.

Negative-Weight Edges

Defining Shortest Paths When There are Negative-Weight Edges

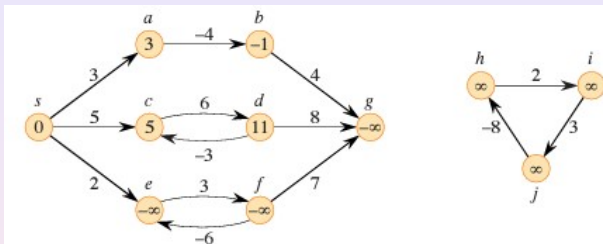
- Some instances of the single-source shortest-paths problem may include edges whose weights are negative.
- If the graph $G = (V, E)$ contains no negative-weight cycles reachable from the source s , then for all $v \in V$, the shortest-path weight $\delta(s, v)$ remains well defined, even if it has a negative value.
- If the graph contains a negative-weight cycle reachable from s , however, shortest-path weights are not well defined.
- No path from s to a vertex on the cycle can be a shortest path since we can always find a path with lower weight by following the proposed "shortest" path and then traversing the negative-weight cycle.

Negative-Weight Edges

Defining Shortest Paths When There are Negative-Weight Edges

- Some instances of the single-source shortest-paths problem may include edges whose weights are negative.
- If the graph $G = (V, E)$ contains no negative-weight cycles reachable from the source s , then for all $v \in V$, the shortest-path weight $\delta(s, v)$ remains well defined, even if it has a negative value.
- If the graph contains a negative-weight cycle reachable from s , however, shortest-path weights are not well defined.
- No path from s to a vertex on the cycle can be a shortest path since we can always find a path with lower weight by following the proposed "shortest" path and then traversing the negative-weight cycle.
- If there is a negative-weight cycle on some path from s to v , we define $\delta(s, v) = -\infty$.

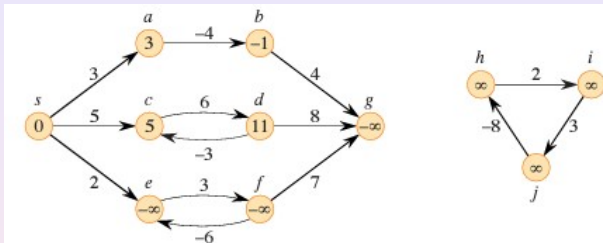
An Example: Shortest Paths in a Graph with Negative-Weight Edges



Explanations

- There are infinitely many paths from s to c : $\langle s, c \rangle, \langle s, c, d, c \rangle, \langle s, c, d, c, d, c \rangle$, and so on. Because the cycle $\langle s, d, c \rangle$ has weight $6 + (-3) = 3 > 0$, the shortest path from s to c is $\langle s, c \rangle$ with weight 5. Similarly, the shortest path from s to d is $\langle s, c, d \rangle$ with weight 11.

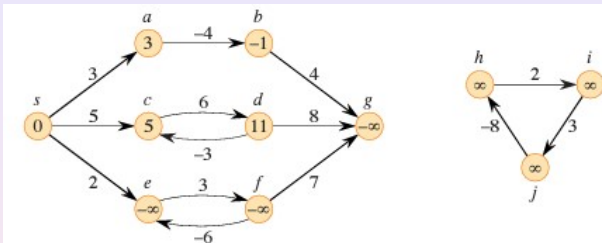
An Example: Shortest Paths in a Graph with Negative-Weight Edges



Explanations

- There are infinitely many paths from s to c : $\langle s, c \rangle, \langle s, c, d, c \rangle, \langle s, c, d, c, d, c \rangle$, and so on. Because the cycle $\langle s, d, c \rangle$ has weight $6 + (-3) = 3 > 0$, the shortest path from s to c is $\langle s, c \rangle$ with weight 5. Similarly, the shortest path from s to d is $\langle s, c, d \rangle$ with weight 11.
- There are infinitely many paths from s to e : $\langle s, e \rangle, \langle s, e, f, e \rangle$, and so on. Because the cycle $\langle e, f, e \rangle$ has weight $3 + (-6) = -3 < 0$, however, there is no shortest path from s to e , and so $\delta(s, e) = -\infty$. Similarly, $\delta(s, f) = -\infty$.

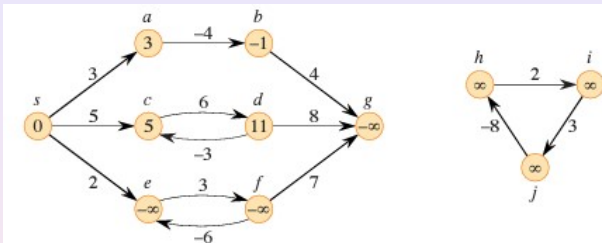
An Example: Shortest Paths in a Graph with Negative-Weight Edges



Explanations

- There are infinitely many paths from s to c : $\langle s, c \rangle, \langle s, c, d, c \rangle, \langle s, c, d, c, d, c \rangle$, and so on. Because the cycle $\langle s, d, c \rangle$ has weight $6 + (-3) = 3 > 0$, the shortest path from s to c is $\langle s, c \rangle$ with weight 5. Similarly, the shortest path from s to d is $\langle s, c, d \rangle$ with weight 11.
- There are infinitely many paths from s to e : $\langle s, e \rangle, \langle s, e, f, e \rangle$, and so on. Because the cycle $\langle e, f, e \rangle$ has weight $3 + (-6) = -3 < 0$, however, there is no shortest path from s to e , and so $\delta(s, e) = -\infty$. Similarly, $\delta(s, f) = -\infty$. Since g is reachable from f , we can find paths with arbitrarily large negative weights from s to g , and so $\delta(s, g) = -\infty$.

An Example: Shortest Paths in a Graph with Negative-Weight Edges



Explanations

- There are infinitely many paths from s to c : $\langle s, c \rangle$, $\langle s, c, d, c \rangle$, $\langle s, c, d, c, d, c \rangle$, and so on. Because the cycle $\langle s, d, c \rangle$ has weight $6 + (-3) = 3 > 0$, the shortest path from s to c is $\langle s, c \rangle$ with weight 5. Similarly, the shortest path from s to d is $\langle s, c, d \rangle$ with weight 11.
- There are infinitely many paths from s to e : $\langle s, e \rangle$, $\langle s, e, f, e \rangle$, and so on. Because the cycle $\langle e, f, e \rangle$ has weight $3 + (-6) = -3 < 0$, however, there is no shortest path from s to e , and so $\delta(s, e) = -\infty$. Similarly, $\delta(s, f) = -\infty$. Since g is reachable from f , we can find paths with arbitrarily large negative weights from s to g , and so $\delta(s, g) = -\infty$.
- Vertices h , i , and j also form a negative-weight cycle. They are not reachable from s , however, and so $\delta(s, h) = \delta(s, i) = \delta(s, j) = \infty$.

Cycles

Shortest Paths are Simple Paths

- As we have just seen, a shortest path cannot contain a negative-weight cycle. Nor can it contain a positive-weight cycle, since removing the cycle from the path produces a path with the same source and destination vertices and a lower path weight.

Cycles

Shortest Paths are Simple Paths

- As we have just seen, a shortest path cannot contain a negative-weight cycle. Nor can it contain a positive-weight cycle, since removing the cycle from the path produces a path with the same source and destination vertices and a lower path weight.
- That leaves only 0-weight cycles. You can remove a 0-weight cycle from any path to produce another path whose weight is the same.

Cycles

Shortest Paths are Simple Paths

- As we have just seen, a shortest path cannot contain a negative-weight cycle. Nor can it contain a positive-weight cycle, since removing the cycle from the path produces a path with the same source and destination vertices and a lower path weight.
- That leaves only 0-weight cycles. You can remove a 0-weight cycle from any path to produce another path whose weight is the same.
- Thus, if there is a shortest path from a source vertex s to a destination vertex v that contains a 0-weight cycle, then there is another shortest path from s to v without this cycle.

Cycles

Shortest Paths are Simple Paths

- As we have just seen, a shortest path cannot contain a negative-weight cycle. Nor can it contain a positive-weight cycle, since removing the cycle from the path produces a path with the same source and destination vertices and a lower path weight.
- That leaves only 0-weight cycles. You can remove a 0-weight cycle from any path to produce another path whose weight is the same.
- Thus, if there is a shortest path from a source vertex s to a destination vertex v that contains a 0-weight cycle, then there is another shortest path from s to v without this cycle.
- As long as a shortest path has 0-weight cycles, you can repeatedly remove these cycles from the path until you have a shortest path that is cycle-free.

Cycles

Shortest Paths are Simple Paths

- As we have just seen, a shortest path cannot contain a negative-weight cycle. Nor can it contain a positive-weight cycle, since removing the cycle from the path produces a path with the same source and destination vertices and a lower path weight.
- That leaves only 0-weight cycles. You can remove a 0-weight cycle from any path to produce another path whose weight is the same.
- Thus, if there is a shortest path from a source vertex s to a destination vertex v that contains a 0-weight cycle, then there is another shortest path from s to v without this cycle.
- As long as a shortest path has 0-weight cycles, you can repeatedly remove these cycles from the path until you have a shortest path that is cycle-free.
- Therefore, without loss of generality, we can assume that shortest paths are simple paths.

Cycles

Shortest Paths are Simple Paths

- As we have just seen, a shortest path cannot contain a negative-weight cycle. Nor can it contain a positive-weight cycle, since removing the cycle from the path produces a path with the same source and destination vertices and a lower path weight.
- That leaves only 0-weight cycles. You can remove a 0-weight cycle from any path to produce another path whose weight is the same.
- Thus, if there is a shortest path from a source vertex s to a destination vertex v that contains a 0-weight cycle, then there is another shortest path from s to v without this cycle.
- As long as a shortest path has 0-weight cycles, you can repeatedly remove these cycles from the path until you have a shortest path that is cycle-free.
- Therefore, without loss of generality, we can assume that shortest paths are simple paths.
- Since any acyclic path in a graph $G = (V, E)$ contains at most $|V|$ distinct vertices, it also contains at most $|V| - 1$ edges.

Cycles

Shortest Paths are Simple Paths

- As we have just seen, a shortest path cannot contain a negative-weight cycle. Nor can it contain a positive-weight cycle, since removing the cycle from the path produces a path with the same source and destination vertices and a lower path weight.
- That leaves only 0-weight cycles. You can remove a 0-weight cycle from any path to produce another path whose weight is the same.
- Thus, if there is a shortest path from a source vertex s to a destination vertex v that contains a 0-weight cycle, then there is another shortest path from s to v without this cycle.
- As long as a shortest path has 0-weight cycles, you can repeatedly remove these cycles from the path until you have a shortest path that is cycle-free.
- Therefore, without loss of generality, we can assume that shortest paths are simple paths.
- Since any acyclic path in a graph $G = (V, E)$ contains at most $|V|$ distinct vertices, it also contains at most $|V| - 1$ edges.
- We will assume, therefore, that any shortest path contains at most $|V| - 1$ edges.

Representing Shortest Paths

Predecessor Subgraph

- It is usually not enough just to compute only shortest-path weights.

Representing Shortest Paths

Predecessor Subgraph

- It is usually not enough just to compute only shortest-path weights.
- You would not use a GPS application that tells you the distance to your destination but not how to get there.

Representing Shortest Paths

Predecessor Subgraph

- It is usually not enough just to compute only shortest-path weights.
- You would not use a GPS application that tells you the distance to your destination but not how to get there.
- We represent shortest paths similarly to how we represented breadth-first trees.

Representing Shortest Paths

Predecessor Subgraph

- It is usually not enough just to compute only shortest-path weights.
- You would not use a GPS application that tells you the distance to your destination but not how to get there.
- We represent shortest paths similarly to how we represented breadth-first trees.
- Given a graph $G = (V, E)$, we maintain for each vertex $v \in V$ a **predecessor** $v.\pi$ that is either another vertex or NIL.

Representing Shortest Paths

Predecessor Subgraph

- It is usually not enough just to compute only shortest-path weights.
- You would not use a GPS application that tells you the distance to your destination but not how to get there.
- We represent shortest paths similarly to how we represented breadth-first trees.
- Given a graph $G = (V, E)$, we maintain for each vertex $v \in V$ a **predecessor** $v.\pi$ that is either another vertex or NIL.
- The shortest-paths algorithms we will see set the π attributes so that the chain of predecessors originating at a vertex v runs backward along a shortest path from s to v .

Representing Shortest Paths

Predecessor Subgraph

- It is usually not enough just to compute only shortest-path weights.
- You would not use a GPS application that tells you the distance to your destination but not how to get there.
- We represent shortest paths similarly to how we represented breadth-first trees.
- Given a graph $G = (V, E)$, we maintain for each vertex $v \in V$ a **predecessor** $v.\pi$ that is either another vertex or NIL.
- The shortest-paths algorithms we will see set the π attributes so that the chain of predecessors originating at a vertex v runs backward along a shortest path from s to v .
- In the midst of executing a shortest-paths algorithm, however, the π values might not indicate shortest paths.

Representing Shortest Paths

Predecessor Subgraph

- It is usually not enough just to compute only shortest-path weights.
- You would not use a GPS application that tells you the distance to your destination but not how to get there.
- We represent shortest paths similarly to how we represented breadth-first trees.
- Given a graph $G = (V, E)$, we maintain for each vertex $v \in V$ a **predecessor** $v.\pi$ that is either another vertex or NIL.
- The shortest-paths algorithms we will see set the π attributes so that the chain of predecessors originating at a vertex v runs backward along a shortest path from s to v .
- In the midst of executing a shortest-paths algorithm, however, the π values might not indicate shortest paths.
- The predecessor subgraph $G_\pi = (V_\pi, E_\pi)$ induced by the π values is defined the same for single-source shortest paths as for breadth-first search:

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}, \text{ and}$$

$$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}.$$

Shortest-Paths Tree

Definition

- The π values produced by the algorithms we will see have the property that at termination G_π is a "shortest-paths tree", i.e., a rooted tree containing a shortest path from the source s to every vertex that is reachable from s .

Shortest-Paths Tree

Definition

- The π values produced by the algorithms we will see have the property that at termination G_π is a "shortest-paths tree", i.e., a rooted tree containing a shortest path from the source s to every vertex that is reachable from s .
- A shortest-paths tree is like the breadth-first tree, but it contains shortest paths from the source defined in terms of edge weights instead of numbers of edges.

Shortest-Paths Tree

Definition

- The π values produced by the algorithms we will see have the property that at termination G_π is a "shortest-paths tree", i.e., a rooted tree containing a shortest path from the source s to every vertex that is reachable from s .
- A shortest-paths tree is like the breadth-first tree, but it contains shortest paths from the source defined in terms of edge weights instead of numbers of edges.
- To be precise, let $G = (V, E)$ be a weighted, directed graph with weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles reachable from the source vertex $s \in V$, so that shortest paths are well defined.

Shortest-Paths Tree

Definition

- The π values produced by the algorithms we will see have the property that at termination G_π is a "shortest-paths tree", i.e., a rooted tree containing a shortest path from the source s to every vertex that is reachable from s .
- A shortest-paths tree is like the breadth-first tree, but it contains shortest paths from the source defined in terms of edge weights instead of numbers of edges.
- To be precise, let $G = (V, E)$ be a weighted, directed graph with weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles reachable from the source vertex $s \in V$, so that shortest paths are well defined.
- A **shortest-paths tree** rooted at s is a directed subgraph $G' = (V', E')$, where $V' \subseteq V$ and $E' \subseteq E$, such that

Shortest-Paths Tree

Definition

- The π values produced by the algorithms we will see have the property that at termination G_π is a "shortest-paths tree", i.e., a rooted tree containing a shortest path from the source s to every vertex that is reachable from s .
- A shortest-paths tree is like the breadth-first tree, but it contains shortest paths from the source defined in terms of edge weights instead of numbers of edges.
- To be precise, let $G = (V, E)$ be a weighted, directed graph with weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles reachable from the source vertex $s \in V$, so that shortest paths are well defined.
- A **shortest-paths tree** rooted at s is a directed subgraph $G' = (V', E')$, where $V' \subseteq V$ and $E' \subseteq E$, such that
 - 1 V' is the set of vertices reachable from s in G ,

Shortest-Paths Tree

Definition

- The π values produced by the algorithms we will see have the property that at termination G_π is a "shortest-paths tree", i.e., a rooted tree containing a shortest path from the source s to every vertex that is reachable from s .
- A shortest-paths tree is like the breadth-first tree, but it contains shortest paths from the source defined in terms of edge weights instead of numbers of edges.
- To be precise, let $G = (V, E)$ be a weighted, directed graph with weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles reachable from the source vertex $s \in V$, so that shortest paths are well defined.
- A **shortest-paths tree** rooted at s is a directed subgraph $G' = (V', E')$, where $V' \subseteq V$ and $E' \subseteq E$, such that
 - 1 V' is the set of vertices reachable from s in G ,
 - 2 G' forms a rooted tree with root s , and

Shortest-Paths Tree

Definition

- The π values produced by the algorithms we will see have the property that at termination G_π is a "shortest-paths tree", i.e., a rooted tree containing a shortest path from the source s to every vertex that is reachable from s .
- A shortest-paths tree is like the breadth-first tree, but it contains shortest paths from the source defined in terms of edge weights instead of numbers of edges.
- To be precise, let $G = (V, E)$ be a weighted, directed graph with weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles reachable from the source vertex $s \in V$, so that shortest paths are well defined.
- A **shortest-paths tree** rooted at s is a directed subgraph $G' = (V', E')$, where $V' \subseteq V$ and $E' \subseteq E$, such that
 - 1 V' is the set of vertices reachable from s in G ,
 - 2 G' forms a rooted tree with root s , and
 - 3 for all $v \in V'$, the unique simple path from s to v in G' is a shortest path from s to v in G .

Shortest-Paths Tree

Definition

- The π values produced by the algorithms we will see have the property that at termination G_π is a "shortest-paths tree", i.e., a rooted tree containing a shortest path from the source s to every vertex that is reachable from s .
- A shortest-paths tree is like the breadth-first tree, but it contains shortest paths from the source defined in terms of edge weights instead of numbers of edges.
- To be precise, let $G = (V, E)$ be a weighted, directed graph with weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles reachable from the source vertex $s \in V$, so that shortest paths are well defined.
- A **shortest-paths tree** rooted at s is a directed subgraph $G' = (V', E')$, where $V' \subseteq V$ and $E' \subseteq E$, such that
 - 1 V' is the set of vertices reachable from s in G ,
 - 2 G' forms a rooted tree with root s , and
 - 3 for all $v \in V'$, the unique simple path from s to v in G' is a shortest path from s to v in G .
- Shortest paths are not necessarily unique, and neither are shortest-paths trees.

Shortest-Paths Trees Are Not Unique

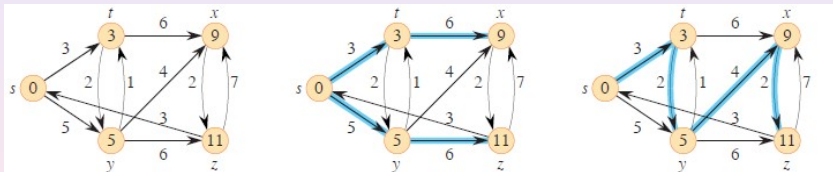


Figure: We have a weighted, directed graph on the left. In the middle, we have the same graph and the blue edges form a shortest-paths tree rooted at the source s . On the right, we have the same graph where the blue edges form another shortest-paths tree with the same root.

Initialization

Initial Values of the Variables

- For each vertex $v \in V$, the single-source shortest paths algorithms maintain an attribute $v.d$, which is an upper bound on the weight of a shortest path from source s to v .

Initialization

Initial Values of the Variables

- For each vertex $v \in V$, the single-source shortest paths algorithms maintain an attribute $v.d$, which is an upper bound on the weight of a shortest path from source s to v .
- We call $v.d$ a shortest-path estimate.

Initialization

Initial Values of the Variables

- For each vertex $v \in V$, the single-source shortest paths algorithms maintain an attribute $v.d$, which is an upper bound on the weight of a shortest path from source s to v .
- We call $v.d$ a shortest-path estimate.
- To initialize the shortest-path estimates and predecessors, we call the $\Theta(n)$ -time procedure INITIALIZE-SINGLE-SOURCE.

Initialization

Initial Values of the Variables

- For each vertex $v \in V$, the single-source shortest paths algorithms maintain an attribute $v.d$, which is an upper bound on the weight of a shortest path from source s to v .
- We call $v.d$ a shortest-path estimate.
- To initialize the shortest-path estimates and predecessors, we call the $\Theta(n)$ -time procedure INITIALIZE-SINGLE-SOURCE.
- After initialization, we have $v.\pi = NIL$ for all $v \in V$, $s.d = 0$, and $v.d = \infty$ for $v \in V - \{s\}$.

Initialization

Initial Values of the Variables

- For each vertex $v \in V$, the single-source shortest paths algorithms maintain an attribute $v.d$, which is an upper bound on the weight of a shortest path from source s to v .
- We call $v.d$ a shortest-path estimate.
- To initialize the shortest-path estimates and predecessors, we call the $\Theta(n)$ -time procedure INITIALIZE-SINGLE-SOURCE.
- After initialization, we have $v.\pi = \text{NIL}$ for all $v \in V$, $s.d = 0$, and $v.d = \infty$ for $v \in V - \{s\}$.

INITIALIZE-SINGLE-SOURCE

INITIALIZE-SINGLE-SOURCE(G, s)

```
1  for each vertex  $v \in G.V$ 
2       $v.d = \infty$ 
3       $v.\pi = \text{NIL}$ 
4   $s.d = 0$ 
```


Relaxation

Relax Operation

- All the algorithms we will see today use the technique called **relaxation**.

Relaxation

Relax Operation

- All the algorithms we will see today use the technique called **relaxation**.
- The process of relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v found so far by going through u and, if so, updating $v.d$ and $v.\pi$.

Relaxation

Relax Operation

- All the algorithms we will see today use the technique called **relaxation**.
- The process of relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v found so far by going through u and, if so, updating $v.d$ and $v.\pi$.
- A relaxation step (which takes $\Theta(1)$ -time) may decrease the value of the shortest path estimate $v.d$ and update v 's predecessor field $v.\pi$.

Relaxation

Relax Operation

- All the algorithms we will see today use the technique called **relaxation**.
- The process of relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v found so far by going through u and, if so, updating $v.d$ and $v.\pi$.
- A relaxation step (which takes $\Theta(1)$ -time) may decrease the value of the shortest path estimate $v.d$ and update v 's predecessor field $v.\pi$.

RELAX(u, v, w)

```
1  if  $v.d > u.d + w(u, v)$   
2       $v.d = u.d + w(u, v)$   
3       $v.\pi = u$ 
```

Relaxation

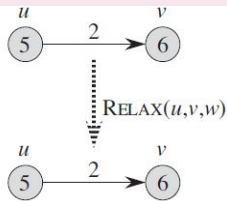
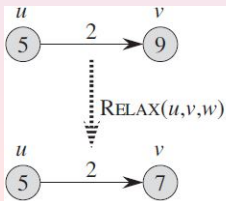
Relax Operation

- All the algorithms we will see today use the technique called **relaxation**.
- The process of relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v found so far by going through u and, if so, updating $v.d$ and $v.\pi$.
- A relaxation step (which takes $\Theta(1)$ -time) may decrease the value of the shortest path estimate $v.d$ and update v 's predecessor field $v.\pi$.

RELAX(u, v, w)

```

1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
  
```



An Outline for our Shortest-Paths Algorithms

Similarities and Differences of Shortest-Paths Algorithms

- All the algorithms ,we will see today call INITIALIZE-SINGLE-SOURCE once as the preprocessing step and then repeatedly relaxes edges.

An Outline for our Shortest-Paths Algorithms

Similarities and Differences of Shortest-Paths Algorithms

- All the algorithms ,we will see today call INITIALIZE-SINGLE-SOURCE once as the preprocessing step and then repeatedly relaxes edges.
- In all of them, relaxation is the only means by which shortest-path estimates and predecessors change.

An Outline for our Shortest-Paths Algorithms

Similarities and Differences of Shortest-Paths Algorithms

- All the algorithms we will see today call INITIALIZE-SINGLE-SOURCE once as the preprocessing step and then repeatedly relaxes edges.
- In all of them, relaxation is the only means by which shortest-path estimates and predecessors change.
- They differ in how many times they relax each edge and the order in which they relax edges.

An Outline for our Shortest-Paths Algorithms

Similarities and Differences of Shortest-Paths Algorithms

- All the algorithms we will see today call INITIALIZE-SINGLE-SOURCE once as the preprocessing step and then repeatedly relaxes edges.
- In all of them, relaxation is the only means by which shortest-path estimates and predecessors change.
- They differ in how many times they relax each edge and the order in which they relax edges.
- Dijkstra's algorithm and the shortest-paths algorithm for directed acyclic graphs relax each edge exactly once.

An Outline for our Shortest-Paths Algorithms

Similarities and Differences of Shortest-Paths Algorithms

- All the algorithms we will see today call INITIALIZE-SINGLE-SOURCE once as the preprocessing step and then repeatedly relaxes edges.
- In all of them, relaxation is the only means by which shortest-path estimates and predecessors change.
- They differ in how many times they relax each edge and the order in which they relax edges.
- Dijkstra's algorithm and the shortest-paths algorithm for directed acyclic graphs relax each edge exactly once.
- The Bellman-Ford algorithm relaxes each edge $|V| - 1$ times.

An Outline for our Shortest-Paths Algorithms

Similarities and Differences of Shortest-Paths Algorithms

- All the algorithms we will see today call INITIALIZE-SINGLE-SOURCE once as the preprocessing step and then repeatedly relaxes edges.
- In all of them, relaxation is the only means by which shortest-path estimates and predecessors change.
- They differ in how many times they relax each edge and the order in which they relax edges.
- Dijkstra's algorithm and the shortest-paths algorithm for directed acyclic graphs relax each edge exactly once.
- The Bellman-Ford algorithm relaxes each edge $|V| - 1$ times.
- To prove the correctness of the algorithms, we'll appeal to 6 properties of shortest paths and relaxation.

An Outline for our Shortest-Paths Algorithms

Similarities and Differences of Shortest-Paths Algorithms

- All the algorithms we will see today call INITIALIZE-SINGLE-SOURCE once as the preprocessing step and then repeatedly relaxes edges.
- In all of them, relaxation is the only means by which shortest-path estimates and predecessors change.
- They differ in how many times they relax each edge and the order in which they relax edges.
- Dijkstra's algorithm and the shortest-paths algorithm for directed acyclic graphs relax each edge exactly once.
- The Bellman-Ford algorithm relaxes each edge $|V| - 1$ times.
- To prove the correctness of the algorithms, we'll appeal to 6 properties of shortest paths and relaxation.
- All our properties (except the first one) implicitly assume that the graph is initialized with a call to INITIALIZE-SINGLE-SOURCE and that the only way that shortest-path estimates and the predecessor subgraph change are by some sequence of relaxation steps.

Properties of Shortest-Paths and Relaxation

Definitions

- **Triangle Inequality:** For any edge $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Properties of Shortest-Paths and Relaxation

Definitions

- **Triangle Inequality:** For any edge $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- **Upper-Bound Property:** We always have $v.d \geq \delta(s, v)$ for all vertices $v \in V$, and once $v.d$ achieves the value $\delta(s, v)$, it never changes.

Properties of Shortest-Paths and Relaxation

Definitions

- **Triangle Inequality:** For any edge $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- **Upper-Bound Property:** We always have $v.d \geq \delta(s, v)$ for all vertices $v \in V$, and once $v.d$ achieves the value $\delta(s, v)$, it never changes.
- **No-path Property:** If there is no path from s to v , we always have $v.d = \delta(s, v) = \infty$.

Properties of Shortest-Paths and Relaxation

Definitions

- **Triangle Inequality:** For any edge $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- **Upper-Bound Property:** We always have $v.d \geq \delta(s, v)$ for all vertices $v \in V$, and once $v.d$ achieves the value $\delta(s, v)$, it never changes.
- **No-path Property:** If there is no path from s to v , we always have $v.d = \delta(s, v) = \infty$.
- **Convergence Property:** If $s \rightsquigarrow u \rightarrow v$ is a shortest path in G for some $u, v \in V$, and if $u.d = \delta(s, u)$ at any time prior to relaxing edge (u, v) , then $v.d = \delta(s, v)$ at all times afterward.

Properties of Shortest-Paths and Relaxation

Definitions

- **Triangle Inequality:** For any edge $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- **Upper-Bound Property:** We always have $v.d \geq \delta(s, v)$ for all vertices $v \in V$, and once $v.d$ achieves the value $\delta(s, v)$, it never changes.
- **No-path Property:** If there is no path from s to v , we always have $v.d = \delta(s, v) = \infty$.
- **Convergence Property:** If $s \rightsquigarrow u \rightarrow v$ is a shortest path in G for some $u, v \in V$, and if $u.d = \delta(s, u)$ at any time prior to relaxing edge (u, v) , then $v.d = \delta(s, v)$ at all times afterward.
- **Path-relaxation Property:** If $p = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from $s = v_0$ to v_k , and the edges of p are relaxed in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, then $v_k.d = \delta(s, v_k)$. This property holds regardless of any other relaxation steps that occur, even if they are intermixed with relaxations of the edges of p .

Properties of Shortest-Paths and Relaxation

Definitions

- **Triangle Inequality:** For any edge $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- **Upper-Bound Property:** We always have $v.d \geq \delta(s, v)$ for all vertices $v \in V$, and once $v.d$ achieves the value $\delta(s, v)$, it never changes.
- **No-path Property:** If there is no path from s to v , we always have $v.d = \delta(s, v) = \infty$.
- **Convergence Property:** If $s \rightsquigarrow u \rightarrow v$ is a shortest path in G for some $u, v \in V$, and if $u.d = \delta(s, u)$ at any time prior to relaxing edge (u, v) , then $v.d = \delta(s, v)$ at all times afterward.
- **Path-relaxation Property:** If $p = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from $s = v_0$ to v_k , and the edges of p are relaxed in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, then $v_k.d = \delta(s, v_k)$. This property holds regardless of any other relaxation steps that occur, even if they are intermixed with relaxations of the edges of p .
- **Predecessor-subgraph Property:** Once $v.d = \delta(s, v)$ for all $v \in V$, the predecessor subgraph is a shortest-paths tree rooted at s .

The Bellman-Ford Algorithm

Explanation

- The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative.

The Bellman-Ford Algorithm

Explanation

- The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
- Given a weighted, directed graph $G = (V, E)$ with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, the Bellman-Ford algorithm returns a boolean value indicating whether there is a negative-weight cycle that is reachable from the source.

The Bellman-Ford Algorithm

Explanation

- The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
- Given a weighted, directed graph $G = (V, E)$ with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, the Bellman-Ford algorithm returns a boolean value indicating whether there is a negative-weight cycle that is reachable from the source.
- The algorithm returns TRUE if and only if the graph contains no negative-weight cycles that are reachable from the source.

The Bellman-Ford Algorithm

Explanation

- The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
- Given a weighted, directed graph $G = (V, E)$ with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, the Bellman-Ford algorithm returns a boolean value indicating whether there is a negative-weight cycle that is reachable from the source.
- The algorithm returns TRUE if and only if the graph contains no negative-weight cycles that are reachable from the source.
- If there is no such cycle, the algorithm produces the shortest paths and their weights.

The Bellman-Ford Algorithm

Explanation

- The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
- Given a weighted, directed graph $G = (V, E)$ with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, the Bellman-Ford algorithm returns a boolean value indicating whether there is a negative-weight cycle that is reachable from the source.
- The algorithm returns TRUE if and only if the graph contains no negative-weight cycles that are reachable from the source.
- If there is no such cycle, the algorithm produces the shortest paths and their weights.
- If there is such a cycle, the algorithm indicates that no solution exists by returning FALSE.

The Bellman-Ford Algorithm

Explanation

- The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
- Given a weighted, directed graph $G = (V, E)$ with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, the Bellman-Ford algorithm returns a boolean value indicating whether there is a negative-weight cycle that is reachable from the source.
- The algorithm returns TRUE if and only if the graph contains no negative-weight cycles that are reachable from the source.
- If there is no such cycle, the algorithm produces the shortest paths and their weights.
- If there is such a cycle, the algorithm indicates that no solution exists by returning FALSE.
- **Explanation of the Algorithm:** The Bellman-Ford algorithm relaxed all the m edges in any order. This way, it guarantees that for any $v \in V$, the first edge of the shortest path from s to v is relaxed. And then it repeats this procedure (relaxing all the edges in any order), $n - 1$ times. The proof of correctness follows via the *path-relaxation property*.

Pseudocode of the Bellman-Ford Algorithm

```
BELLMAN-FORD( $G, w, s$ )  
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )  
2  for  $i = 1$  to  $|G.V| - 1$   
3      for each edge  $(u, v) \in G.E$   
4          RELAX( $u, v, w$ )  
5  for each edge  $(u, v) \in G.E$   
6      if  $v.d > u.d + w(u, v)$   
7          return FALSE  
8  return TRUE
```

Pseudocode of the Bellman-Ford Algorithm

```
BELLMAN-FORD( $G, w, s$ )  
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )  
2  for  $i = 1$  to  $|G.V| - 1$   
3      for each edge  $(u, v) \in G.E$   
4          RELAX( $u, v, w$ )  
5  for each edge  $(u, v) \in G.E$   
6      if  $v.d > u.d + w(u, v)$   
7          return FALSE  
8  return TRUE
```

Running Time Analysis

The inner for loop takes $\Theta(n + m)$ -time, and it is repeated $n - 1$ times, therefore, the algorithm runs in time $\Theta(n^2 + nm)$. In the frequent case, where $m = \Omega(n)$, we can express this running time as $\Theta(mn)$.

Bellman-Ford Proof Of Correctness: No Negative-Cycle Case

Lemma

Let $G = (V, E)$ be a weighted, directed graph with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles that are reachable from s . Then, after the $|V| - 1$ iterations of the for loop of lines 2 – 4 of BELLMAN-FORD, $v.d = \delta(s, v)$ for all vertices v .

Bellman-Ford Proof Of Correctness: No Negative-Cycle Case

Lemma

Let $G = (V, E)$ be a weighted, directed graph with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles that are reachable from s . Then, after the $|V| - 1$ iterations of the for loop of lines 2 – 4 of BELLMAN-FORD, $v.d = \delta(s, v)$ for all vertices v .

Proof

- We prove the lemma by appealing to the path-relaxation and no-path properties.

Bellman-Ford Proof Of Correctness: No Negative-Cycle Case

Lemma

Let $G = (V, E)$ be a weighted, directed graph with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles that are reachable from s . Then, after the $|V| - 1$ iterations of the for loop of lines 2 – 4 of BELLMAN-FORD, $v.d = \delta(s, v)$ for all vertices v .

Proof

- We prove the lemma by appealing to the path-relaxation and no-path properties.
- Consider any vertex v that is reachable from s , and let $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$, be any shortest path from s to v .

Bellman-Ford Proof Of Correctness: No Negative-Cycle Case

Lemma

Let $G = (V, E)$ be a weighted, directed graph with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles that are reachable from s . Then, after the $|V| - 1$ iterations of the for loop of lines 2 – 4 of BELLMAN-FORD, $v.d = \delta(s, v)$ for all vertices v .

Proof

- We prove the lemma by appealing to the path-relaxation and no-path properties.
- Consider any vertex v that is reachable from s , and let $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$, be any shortest path from s to v .
- Because shortest paths are simple, p has at most $|V| - 1$ edges, and so $k \leq |V| - 1$.

Bellman-Ford Proof Of Correctness: No Negative-Cycle Case

Lemma

Let $G = (V, E)$ be a weighted, directed graph with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles that are reachable from s . Then, after the $|V| - 1$ iterations of the for loop of lines 2 – 4 of BELLMAN-FORD, $v.d = \delta(s, v)$ for all vertices v .

Proof

- We prove the lemma by appealing to the path-relaxation and no-path properties.
- Consider any vertex v that is reachable from s , and let $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$, be any shortest path from s to v .
- Because shortest paths are simple, p has at most $|V| - 1$ edges, and so $k \leq |V| - 1$.
- Each of the $|V| - 1$ iterations of the outer for loop of lines 2 – 4 relaxes all $|E|$ edges.

Bellman-Ford Proof Of Correctness: No Negative-Cycle Case

Lemma

Let $G = (V, E)$ be a weighted, directed graph with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles that are reachable from s . Then, after the $|V| - 1$ iterations of the for loop of lines 2 – 4 of BELLMAN-FORD, $v.d = \delta(s, v)$ for all vertices v .

Proof

- We prove the lemma by appealing to the path-relaxation and no-path properties.
- Consider any vertex v that is reachable from s , and let $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$, be any shortest path from s to v .
- Because shortest paths are simple, p has at most $|V| - 1$ edges, and so $k \leq |V| - 1$.
- Each of the $|V| - 1$ iterations of the outer for loop of lines 2 – 4 relaxes all $|E|$ edges.
- Among the edges relaxed in iteration i (for $i = 1, 2, \dots, k$) is (v_{i-1}, v_i) .

Bellman-Ford Proof Of Correctness: No Negative-Cycle Case

Lemma

Let $G = (V, E)$ be a weighted, directed graph with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles that are reachable from s . Then, after the $|V| - 1$ iterations of the for loop of lines 2 – 4 of BELLMAN-FORD, $v.d = \delta(s, v)$ for all vertices v .

Proof

- We prove the lemma by appealing to the path-relaxation and no-path properties.
- Consider any vertex v that is reachable from s , and let $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$, be any shortest path from s to v .
- Because shortest paths are simple, p has at most $|V| - 1$ edges, and so $k \leq |V| - 1$.
- Each of the $|V| - 1$ iterations of the outer for loop of lines 2 – 4 relaxes all $|E|$ edges.
- Among the edges relaxed in iteration i (for $i = 1, 2, \dots, k$) is (v_{i-1}, v_i) .
- By the path-relaxation property, therefore, $v.d = v_k.d = \delta(s, v_k) = \delta(s, v)$.

Bellman-Ford Proof Of Correctness: No Negative-Cycle Case

Lemma

Let $G = (V, E)$ be a weighted, directed graph with source vertex s and weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles that are reachable from s . Then, after the $|V| - 1$ iterations of the for loop of lines 2 – 4 of BELLMAN-FORD, $v.d = \delta(s, v)$ for all vertices v .

Proof

- We prove the lemma by appealing to the path-relaxation and no-path properties.
- Consider any vertex v that is reachable from s , and let $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$, be any shortest path from s to v .
- Because shortest paths are simple, p has at most $|V| - 1$ edges, and so $k \leq |V| - 1$.
- Each of the $|V| - 1$ iterations of the outer for loop of lines 2 – 4 relaxes all $|E|$ edges.
- Among the edges relaxed in iteration i (for $i = 1, 2, \dots, k$) is (v_{i-1}, v_i) .
- By the path-relaxation property, therefore, $v.d = v_k.d = \delta(s, v_k) = \delta(s, v)$.
- If v is not reachable from s , then the claim follows from the no-path property.

Bellman-Ford Proof Of Correctness: Negative-Cycle Case

Lemma

Bellman-Ford algorithm returns FALSE if G contains a negative-weight cycle reachable from s .

Bellman-Ford Proof Of Correctness: Negative-Cycle Case

Lemma

Bellman-Ford algorithm returns FALSE if G contains a negative-weight cycle reachable from s .

Proof

- Suppose that G contains a negative-weight cycle reachable from s , and let this cycle be $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$, in which case we have $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$.

Bellman-Ford Proof Of Correctness: Negative-Cycle Case

Lemma

Bellman-Ford algorithm returns FALSE if G contains a negative-weight cycle reachable from s .

Proof

- Suppose that G contains a negative-weight cycle reachable from s , and let this cycle be $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$, in which case we have $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$.
- Assume for the purpose of contradiction that the Bellman-Ford algorithm returns TRUE. Thus, $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ for $i = 1, 2, \dots, k$.

Bellman-Ford Proof Of Correctness: Negative-Cycle Case

Lemma

Bellman-Ford algorithm returns FALSE if G contains a negative-weight cycle reachable from s .

Proof

- Suppose that G contains a negative-weight cycle reachable from s , and let this cycle be $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$, in which case we have $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$.
- Assume for the purpose of contradiction that the Bellman-Ford algorithm returns TRUE. Thus, $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ for $i = 1, 2, \dots, k$.
- Summing the inequalities around cycle c gives:

$$\sum_{i=1}^k v_i.d \leq \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i)) = \sum_{i=1}^k v_{i-1}.d + \sum_{i=1}^k w(v_{i-1}, v_i)$$

Bellman-Ford Proof Of Correctness: Negative-Cycle Case

Lemma

Bellman-Ford algorithm returns FALSE if G contains a negative-weight cycle reachable from s .

Proof

- Suppose that G contains a negative-weight cycle reachable from s , and let this cycle be $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$, in which case we have $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$.
- Assume for the purpose of contradiction that the Bellman-Ford algorithm returns TRUE. Thus, $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ for $i = 1, 2, \dots, k$.
- Summing the inequalities around cycle c gives:

$$\sum_{i=1}^k v_i.d \leq \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i)) = \sum_{i=1}^k v_{i-1}.d + \sum_{i=1}^k w(v_{i-1}, v_i)$$

- Since $v_0 = v_k$, each vertex in c appears exactly once in each of the summations $\sum_{i=1}^k v_i.d$ and $\sum_{i=1}^k v_{i-1}.d$, and so $\sum_{i=1}^k v_i.d = \sum_{i=1}^k v_{i-1}.d$.

Bellman-Ford Proof Of Correctness: Negative-Cycle Case

Lemma

Bellman-Ford algorithm returns FALSE if G contains a negative-weight cycle reachable from s .

Proof

- Suppose that G contains a negative-weight cycle reachable from s , and let this cycle be $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$, in which case we have $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$.
- Assume for the purpose of contradiction that the Bellman-Ford algorithm returns TRUE. Thus, $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ for $i = 1, 2, \dots, k$.
- Summing the inequalities around cycle c gives:

$$\sum_{i=1}^k v_i.d \leq \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i)) = \sum_{i=1}^k v_{i-1}.d + \sum_{i=1}^k w(v_{i-1}, v_i)$$

- Since $v_0 = v_k$, each vertex in c appears exactly once in each of the summations $\sum_{i=1}^k v_i.d$ and $\sum_{i=1}^k v_{i-1}.d$, and so $\sum_{i=1}^k v_i.d = \sum_{i=1}^k v_{i-1}.d$.
- Thus, $0 \leq \sum_{i=1}^k w(v_{i-1}, v_i)$, which contradicts c being a negative-weight cycle.

Bellman-Ford Proof Of Correctness: Negative-Cycle Case

Lemma

Bellman-Ford algorithm returns FALSE if G contains a negative-weight cycle reachable from s .

Proof

- Suppose that G contains a negative-weight cycle reachable from s , and let this cycle be $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$, in which case we have $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$.
- Assume for the purpose of contradiction that the Bellman-Ford algorithm returns TRUE. Thus, $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ for $i = 1, 2, \dots, k$.
- Summing the inequalities around cycle c gives:

$$\sum_{i=1}^k v_i.d \leq \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i)) = \sum_{i=1}^k v_{i-1}.d + \sum_{i=1}^k w(v_{i-1}, v_i)$$

- Since $v_0 = v_k$, each vertex in c appears exactly once in each of the summations $\sum_{i=1}^k v_i.d$ and $\sum_{i=1}^k v_{i-1}.d$, and so $\sum_{i=1}^k v_i.d = \sum_{i=1}^k v_{i-1}.d$.
- Thus, $0 \leq \sum_{i=1}^k w(v_{i-1}, v_i)$, which contradicts c being a negative-weight cycle.
- Hence, Bellman-Ford algorithm returns FALSE if graph G contains negative-weight cycles reachable from s .

Shortest Paths in Directed Acyclic Graphs

Explanation of the Algorithm

- Shortest paths are always well-defined in DAGs since even if there is a negative-weight edge there cannot be any negative-weight cycle.

Shortest Paths in Directed Acyclic Graphs

Explanation of the Algorithm

- Shortest paths are always well-defined in DAGs since even if there is a negative-weight edge there cannot be any negative-weight cycle.
- If the edges of a weighted dag $G = (V, E)$ are relaxed according to a topological sort of its vertices, it takes only $\Theta(n + m)$ time to compute shortest paths from a single source.

Shortest Paths in Directed Acyclic Graphs

Explanation of the Algorithm

- Shortest paths are always well-defined in DAGs since even if there is a negative-weight edge there cannot be any negative-weight cycle.
- If the edges of a weighted dag $G = (V, E)$ are relaxed according to a topological sort of its vertices, it takes only $\Theta(n + m)$ time to compute shortest paths from a single source.

DAG-SHORTEST-PATHS(G, w, s)

```
1  topologically sort the vertices of  $G$ 
2  INITIALIZE-SINGLE-SOURCE( $G, s$ )
3  for each vertex  $u$ , taken in topologically sorted order
4      for each vertex  $v \in G.Adj[u]$ 
5          RELAX( $u, v, w$ )
```

Proof Of Correctness of the Algorithm

Theorem

If a weighted, directed graph $G = (V, E)$ has source vertex s and no cycles, then at the termination of the DAG-SHORTEST-PATHS procedure, $v.d = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree.

Proof Of Correctness of the Algorithm

Theorem

If a weighted, directed graph $G = (V, E)$ has source vertex s and no cycles, then at the termination of the DAG-SHORTEST-PATHS procedure, $v.d = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree.

Proof

- We first show that $v.d = \delta(s, v)$ for all vertices $v \in V$ at termination.

Proof Of Correctness of the Algorithm

Theorem

If a weighted, directed graph $G = (V, E)$ has source vertex s and no cycles, then at the termination of the DAG-SHORTEST-PATHS procedure, $v.d = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree.

Proof

- We first show that $v.d = \delta(s, v)$ for all vertices $v \in V$ at termination.
- If v is not reachable from s , then $v.d = \delta(s, v) = \infty$ by the no-path property.

Proof Of Correctness of the Algorithm

Theorem

If a weighted, directed graph $G = (V, E)$ has source vertex s and no cycles, then at the termination of the DAG-SHORTEST-PATHS procedure, $v.d = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree.

Proof

- We first show that $v.d = \delta(s, v)$ for all vertices $v \in V$ at termination.
- If v is not reachable from s , then $v.d = \delta(s, v) = \infty$ by the no-path property.
- Now, suppose that v is reachable from s , so that there is a shortest path $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$.

Proof Of Correctness of the Algorithm

Theorem

If a weighted, directed graph $G = (V, E)$ has source vertex s and no cycles, then at the termination of the DAG-SHORTEST-PATHS procedure, $v.d = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree.

Proof

- We first show that $v.d = \delta(s, v)$ for all vertices $v \in V$ at termination.
- If v is not reachable from s , then $v.d = \delta(s, v) = \infty$ by the no-path property.
- Now, suppose that v is reachable from s , so that there is a shortest path $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$.
- Because DAG-SHORTEST-PATHS processes the vertices in topologically sorted order, it relaxes the edges on p in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$.

Proof Of Correctness of the Algorithm

Theorem

If a weighted, directed graph $G = (V, E)$ has source vertex s and no cycles, then at the termination of the DAG-SHORTEST-PATHS procedure, $v.d = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree.

Proof

- We first show that $v.d = \delta(s, v)$ for all vertices $v \in V$ at termination.
- If v is not reachable from s , then $v.d = \delta(s, v) = \infty$ by the no-path property.
- Now, suppose that v is reachable from s , so that there is a shortest path $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$.
- Because DAG-SHORTEST-PATHS processes the vertices in topologically sorted order, it relaxes the edges on p in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$.
- The path-relaxation property implies that $v_i.d = \delta(s, v_i)$ at termination for $i = 0, 1, \dots, k$.

Proof Of Correctness of the Algorithm

Theorem

If a weighted, directed graph $G = (V, E)$ has source vertex s and no cycles, then at the termination of the DAG-SHORTEST-PATHS procedure, $v.d = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree.

Proof

- We first show that $v.d = \delta(s, v)$ for all vertices $v \in V$ at termination.
- If v is not reachable from s , then $v.d = \delta(s, v) = \infty$ by the no-path property.
- Now, suppose that v is reachable from s , so that there is a shortest path $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$.
- Because DAG-SHORTEST-PATHS processes the vertices in topologically sorted order, it relaxes the edges on p in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$.
- The path-relaxation property implies that $v_i.d = \delta(s, v_i)$ at termination for $i = 0, 1, \dots, k$.
- Finally, by the predecessor-subgraph property, G_π is a shortest-paths tree.

Dijkstra's Algorithm

An Overview

- Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$, but it requires nonnegative weights on all edges.

Dijkstra's Algorithm

An Overview

- Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$, but it requires nonnegative weights on all edges.
- Its running time is lower than that of the Bellman-Ford algorithm.

Dijkstra's Algorithm

An Overview

- Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$, but it requires nonnegative weights on all edges.
- Its running time is lower than that of the Bellman-Ford algorithm.
- You can think of Dijkstra's algorithm as generalizing BFS to weighted graphs.

Dijkstra's Algorithm

An Overview

- Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$, but it requires nonnegative weights on all edges.
- Its running time is lower than that of the Bellman-Ford algorithm.
- You can think of Dijkstra's algorithm as generalizing BFS to weighted graphs.
- A wave emanates from the source, and the first time that a wave arrives at a vertex, a new wave emanates from that vertex. Whereas BFS operates as if each wave takes unit time to traverse an edge, in a weighted graph, the time for a wave to traverse an edge is given by the edge's weight.

Dijkstra's Algorithm

An Overview

- Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$, but it requires nonnegative weights on all edges.
- Its running time is lower than that of the Bellman-Ford algorithm.
- You can think of Dijkstra's algorithm as generalizing BFS to weighted graphs.
- A wave emanates from the source, and the first time that a wave arrives at a vertex, a new wave emanates from that vertex. Whereas BFS operates as if each wave takes unit time to traverse an edge, in a weighted graph, the time for a wave to traverse an edge is given by the edge's weight.
- Because a shortest path in a weighted graph might not have the fewest edges, a simple, FIFO queue won't suffice for choosing the next vertex from which to send out a wave.

Dijkstra's Algorithm

An Overview

- Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$, but it requires nonnegative weights on all edges.
- Its running time is lower than that of the Bellman-Ford algorithm.
- You can think of Dijkstra's algorithm as generalizing BFS to weighted graphs.
- A wave emanates from the source, and the first time that a wave arrives at a vertex, a new wave emanates from that vertex. Whereas BFS operates as if each wave takes unit time to traverse an edge, in a weighted graph, the time for a wave to traverse an edge is given by the edge's weight.
- Because a shortest path in a weighted graph might not have the fewest edges, a simple, FIFO queue won't suffice for choosing the next vertex from which to send out a wave.
- Instead, Dijkstra's algorithm maintains a set S of vertices whose final shortest-path weights from the source s have already been determined.

Dijkstra's Algorithm

An Overview

- Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$, but it requires nonnegative weights on all edges.
- Its running time is lower than that of the Bellman-Ford algorithm.
- You can think of Dijkstra's algorithm as generalizing BFS to weighted graphs.
- A wave emanates from the source, and the first time that a wave arrives at a vertex, a new wave emanates from that vertex. Whereas BFS operates as if each wave takes unit time to traverse an edge, in a weighted graph, the time for a wave to traverse an edge is given by the edge's weight.
- Because a shortest path in a weighted graph might not have the fewest edges, a simple, FIFO queue won't suffice for choosing the next vertex from which to send out a wave.
- Instead, Dijkstra's algorithm maintains a set S of vertices whose final shortest-path weights from the source s have already been determined.
- The algorithm repeatedly selects the vertex $u \in V - S$ with the minimum shortest-path estimate, adds u into S , and relaxes all edges leaving u .

Dijkstra's Algorithm

An Overview

- Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$, but it requires nonnegative weights on all edges.
- Its running time is lower than that of the Bellman-Ford algorithm.
- You can think of Dijkstra's algorithm as generalizing BFS to weighted graphs.
- A wave emanates from the source, and the first time that a wave arrives at a vertex, a new wave emanates from that vertex. Whereas BFS operates as if each wave takes unit time to traverse an edge, in a weighted graph, the time for a wave to traverse an edge is given by the edge's weight.
- Because a shortest path in a weighted graph might not have the fewest edges, a simple, FIFO queue won't suffice for choosing the next vertex from which to send out a wave.
- Instead, Dijkstra's algorithm maintains a set S of vertices whose final shortest-path weights from the source s have already been determined.
- The algorithm repeatedly selects the vertex $u \in V - S$ with the minimum shortest-path estimate, adds u into S , and relaxes all edges leaving u .
- Dijkstra's algorithm replaces the FIFO queue of BFS by a min-priority queue Q of vertices, keyed by their d values.

Pseudocode for Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = \emptyset$ 
4  for each vertex  $u \in G.V$ 
5      INSERT( $Q, u$ )
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8       $S = S \cup \{u\}$ 
9      for each vertex  $v$  in  $G.Adj[u]$ 
10         RELAX( $u, v, w$ )
11         if the call of RELAX decreased  $v.d$ 
12             DECREASE-KEY( $Q, v, v.d$ )
```

Running Time of Dijkstra's Algorithm

Discussing Implementation Strategies

- The running-time of Dijkstra's algorithm is $\Theta(n + m)$ plus the time spend on priority queue operations INSERT, EXTRACT-MIN, and DECREASE-KEY.

Running Time of Dijkstra's Algorithm

Discussing Implementation Strategies

- The running-time of Dijkstra's algorithm is $\Theta(n + m)$ plus the time spend on priority queue operations INSERT, EXTRACT-MIN, and DECREASE-KEY.
- Algorithm calls both INSERT and and EXTRACT-MIN once per vertex, and so we call each of these operations n times.

Running Time of Dijkstra's Algorithm

Discussing Implementation Strategies

- The running-time of Dijkstra's algorithm is $\Theta(n + m)$ plus the time spend on priority queue operations INSERT, EXTRACT-MIN, and DECREASE-KEY.
- Algorithm calls both INSERT and and EXTRACT-MIN once per vertex, and so we call each of these operations n times.
- Since each edge is relaxed only once, we call DECREASE-KEY operation m times.

Running Time of Dijkstra's Algorithm

Discussing Implementation Strategies

- The running-time of Dijkstra's algorithm is $\Theta(n + m)$ plus the time spend on priority queue operations INSERT, EXTRACT-MIN, and DECREASE-KEY.
- Algorithm calls both INSERT and and EXTRACT-MIN once per vertex, and so we call each of these operations n times.
- Since each edge is relaxed only once, we call DECREASE-KEY operation m times.
- The running time of Dijkstra's algorithm depends on the priority queue implementation.

Running Time of Dijkstra's Algorithm

Discussing Implementation Strategies

- The running-time of Dijkstra's algorithm is $\Theta(n + m)$ plus the time spend on priority queue operations INSERT, EXTRACT-MIN, and DECREASE-KEY.
- Algorithm calls both INSERT and and EXTRACT-MIN once per vertex, and so we call each of these operations n times.
- Since each edge is relaxed only once, we call DECREASE-KEY operation m times.
- The running time of Dijkstra's algorithm depends on the priority queue implementation.
- If we maintain the priority queue as a binary heap, then each INSERT, EXTRACT-MIN and DECREASE-KEY operation takes $O(\lg n)$ -time and thus, total running time is $O(m \lg n)$.

Running Time of Dijkstra's Algorithm

Discussing Implementation Strategies

- The running-time of Dijkstra's algorithm is $\Theta(n + m)$ plus the time spend on priority queue operations INSERT, EXTRACT-MIN, and DECREASE-KEY.
- Algorithm calls both INSERT and and EXTRACT-MIN once per vertex, and so we call each of these operations n times.
- Since each edge is relaxed only once, we call DECREASE-KEY operation m times.
- The running time of Dijkstra's algorithm depends on the priority queue implementation.
- If we maintain the priority queue as a binary heap, then each INSERT, EXTRACT-MIN and DECREASE-KEY operation takes $O(\lg n)$ -time and thus, total running time is $O(m \lg n)$.
- We can instead maintain the min-priority queue by taking advantage of the vertices numbered from 1 to n , and store the value $v.d$ at index v of an array. Then, each INSERT and DECREASE-KEY operation takes $O(1)$ -time. EXTRACT-MIN, however, takes $O(n)$ -time. Since throughout the execution of the algorithm, we have a total of n INSERT, m DECREASE-KEY, and n EXTRACT-MIN operations, this takes a total time of $\Theta(n^2)$, provided that $m = \Omega(n)$.

Running Time of Dijkstra's Algorithm

Discussing Implementation Strategies

- The running-time of Dijkstra's algorithm is $\Theta(n + m)$ plus the time spend on priority queue operations INSERT, EXTRACT-MIN, and DECREASE-KEY.
- Algorithm calls both INSERT and and EXTRACT-MIN once per vertex, and so we call each of these operations n times.
- Since each edge is relaxed only once, we call DECREASE-KEY operation m times.
- The running time of Dijkstra's algorithm depends on the priority queue implementation.
- If we maintain the priority queue as a binary heap, then each INSERT, EXTRACT-MIN and DECREASE-KEY operation takes $O(\lg n)$ -time and thus, total running time is $O(m \lg n)$.
- We can instead maintain the min-priority queue by taking advantage of the vertices numbered from 1 to n , and store the value $v.d$ at index v of an array. Then, each INSERT and DECREASE-KEY operation takes $O(1)$ -time. EXTRACT-MIN, however, takes $O(n)$ -time. Since throughout the execution of the algorithm, we have a total of n INSERT, m DECREASE-KEY, and n EXTRACT-MIN operations, this takes a total time of $\Theta(n^2)$, provided that $m = \Omega(n)$.
- Notice that binary min-heap based implementation is better than $O(n^2)$ if the graph is sparse enough, i.e., $m = o(\frac{n^2}{\lg n})$.